

Технологии программирования

gRPC

Раздел 1

RPC как стиль проектирования API

Что такое RPC с точки зрения архитектуры

RPC (Remote Procedure Call) в первую очередь описывает способ мышления об удалённом взаимодействии: вы хотите вызывать операции удалённой системы так, как если бы это были методы или функции в этой же системе.

При этом важно не путать абстракцию и реальность: при взаимодействии по сети существуют задержки, частичные отказы и ограниченная надёжность.

Как стиль, RPC обычно проявляется в том, что публичный контракт выражается через набор операций, (процедур), и структур данных, которые эти операции принимают и возвращают. Это ближе к «сделай X» и «получи Y», чем к модели «ресурс R в состоянии S», которая характерна для REST.

Важно:

RPC – это не конкретный протокол, а семейство подходов.

А вот gRPC – это именно реализация RPC, одна из многих.

Что именно в RPC фиксируется контрактом

В RPC-стиле контракт явно фиксирует:

- **имена операций;**
- **аргументы операций;**
- **результаты операций.**

Это делает интеграцию более «жёсткой»: клиент и сервер обязаны согласовать сигнатуры, иначе взаимодействие просто не соберётся. Но это требование исключает рассинхронизации между ними, в отличие от REST с его JSON`ами.

Ещё одно практическое следствие RPC-мышления: вы чаще проектируете систему как набор сервисов с чёткими границами и явными зависимостями, потому что каждая операция – это отдельная точка сопряжения, которую нужно сопровождать и тестировать.

Итого:

- Стил RPC – это набор «методов» системы, через которые к ней можно обратиться.
- Общий контракт создает сильную связь между клиентом и сервером, нарушить которую невозможно.

Раздел 2

Сравнение REST и RPC

REST: ресурсы, представления и HTTP-семантика

REST предлагает думать о системе, как о наборе ресурсов, идентифицируемых URI, и операций над ними через ограниченный набор глаголов HTTP. Смысл эндпоинта заключается в комбинации ресурса и одного из методов: GET, POST, PUT, PATCH или DELETE, (сильно реже: OPTIONS, HEAD, TRACE, CONNECT).

Сильная сторона REST — унификация и предсказуемость для широкого круга клиентов, особенно когда API публичный и живёт в браузерной экосистеме. Также REST хорошо сочетается с кэшированием HTTP, если модель ресурсов позволяет применять кэш осмысленно.

REST: требования к сообщениям

В REST есть единственное жесткое требование к сообщениям: они должны быть самоописываемыми, то есть все сведения о том, как обрабатывать сообщение содержатся в самом сообщении.

Можно возразить: описание HTTP – статусов и заголовков сообщений не хранится в сообщении. Например, нам приходит ответ: 200 OK с заголовком: Content-type: application/json.

Но информация о значении каждого HTTP – статуса и каждого заголовка хранится в различных стандартах соответствующей технологии, то есть имеют однозначную и единственную трактовку. А любые API, фреймворки, инструменты, библиотеки должны соблюдать эти стандарты, если хотят быть RESTful.

Подытоживая, самоописываемым сообщением может называться только то, для расшифровки которого, не нужно читать документацию конкретного API. Все, что нам нужно для его прочтения либо находится в нем, либо в стандартах технологий, которые в этом сообщении используются.

Для кругозора:

Стандарт – официальный технический документ, описывающий технологию и как с ней взаимодействовать.

RFC 7230 – стандарт, описывающий HTTP/1.1.

RFC 8259 – стандарт, описывающий JSON.

RPC: операции и явные контракты

RPC чаще формулируется, как «вызови операцию X с аргументами A и получи результат B».

Это ближе к тому, как разработчики описывают бизнес-действия внутри сервиса: списать баланс, проверить квоту, пересчитать отчёт. Поэтому RPC часто естественнее для внутренних API между сервисами, где нет требования «выглядеть как CRUD над ресурсами».

С точки зрения эволюции API, RPC обычно требует более дисциплинированного подхода к версиям контракта: вы не можете расширить контракт, добавив поле в ответ, не обновив клиентов.

Если изменили сигнатуру операции или семантику запроса/ответа, будьте добры или оставить поддержку старой версии или обновить абсолютно всех клиентов, обращающихся к вашему API.

RPC: требования к сообщениям

Исходя из строгости, которую накладывают на синхронизацию сигнатуры операции, запроса и ответа между клиентом и сервером, в RPC распространен подход с генерацией вышеперечисленных из описания API.

А описание API в свою очередь пишется с помощью RPC фреймворков, созданных для этой цели, например Thrift или Protocol Buffers.

То есть для реализации взаимодействия между клиентом и сервером через RPC, необходимо сначала описать API, (операцию, запрос, ответ), затем сгенерировать на основе этого контракта сообщения и реализации клиента и сервера для нужного вам языка программирования.

Это является еще одним кардинальным отличием RPC от REST:

- нет необходимости в самоописываемых сообщениях, в которых используется только общеизвестная информация, потому что все клиенты и все серверы генерируются из контракта, из которого понятно, как эти сообщения RPC обрабатывать;
- возможностей для полета фантазии при разработке реализации еще меньше: если контракт не будет соблюден, процесс упадет с внутренней ошибкой и ничего никуда не отправится.

REST vs RPC

	RPC	REST
Причина создания	Необходимость как-то пересылать информацию по сети. Похожа идеологически на модель взаимодействия внутри систем.	Необходимость унифицировать, (в какой-то степени) огромное множество API в стиле RPC.
Действующие лица	Операция, запрос, ответ.	Ресурс и HTTP – метод.
Ограничения	Грубо говоря, отсутствуют. Каждый разработчик делает так, как посчитает нужным, максимум опираясь на best practices.	Есть требования к: наименованию эндпоинтов, форматам сообщений, используемым протоколам, кодам ошибок и т.д.

Раздел 3

Проблемы и ограничения REST

JSON и текстовые форматы

JSON удобен человеку, но при высокой нагрузке он часто становится узким местом: растёт объём трафика, время необходимое на сериализацию/десериализацию и нагрузка на CPU.

JSON – не плохо, но когда объём сообщений велик, трата байт и CPU начинает сказываться на времени отклика, а следовательно удовлетворенности пользователей системы.

Также JSON слабее, как контракт, без дополнительных инструментов (JSON Schema, OpenAPI) очень быстро API скатывается к ситуации, когда клиент и сервер понимают поля по-разному.

OpenAPI помогает, но поддержка схемы и реализации в синхронизованном состоянии – отдельная организационная, (трудоемкая и неблагодарная) задача.

Важно:

JSON – не ужасно и не страшно, (как и других текстовых форматов), их можно и нужно использовать.

Но нужно помнить: волшебной палочки не существует, для разных задач – разные инструменты.

Контракт

При проектировании REST API отсутствует возможность жестко задать контракт взаимодействия между клиентом и сервером.

То есть, описать запрос, ответ и ошибки можно, но уверенности что эндпоинт будет работать так, как описано, нет. Это происходит потому что жесткой связки между описанием и реализацией нет.

У разработчика есть возможность написать в OpenAPI, (Swagger), все, что угодно, потому что связь с кодом у инструмента весьма условная.

Но даже отбросив намеренное ошибочное написание документации к API, очень часто случается так, что физически API как-то изменили, добавили новое поле в ответ, например, а добавить его контракт забыли. А после нескольких подобных итераций, документацию можно считать устаревшей, и проще ее писать заново, чем пытаться синхронизировать с текущим состоянием API.

Но необходимость использовать API осталась, и его продолжают использовать, но сталкиваются с ответами, которые отличны от того, чего ожидали.

Итого:

- REST обладает рядом недостатков, (мы обсудили не все, а самые критичные).
- Эти недостатки становятся существенными в высоконагруженных системах.

Раздел 4

Что такое gRPC

Что такое gRPC

gRPC – это фреймворк и экосистема для RPC, где контракт описывается в Protocol Buffers, а в качестве транспортного протокола по умолчанию используется HTTP/2.

Разработчик получает:

- строгие сообщения;
- генерацию кода клиента/сервера;
- бинарный компактный wire-формат;
- модель RPC, включая unary и несколько видов streaming.

gRPC чаще всего используется для внутреннего взаимодействия между сервисами.

Для публичного «браузерного» API он не всегда удобен без дополнительных слоёв (grpc-web, шлюзы), поэтому в продукте часто встречается гибрид: снаружи REST, внутри gRPC.

Из чего состоит gRPC-проект

- Почти всегда есть .proto файлы, которые описывают сервисы и сообщения. По ним будут сгенерированы классы сообщений и stub-интерфейсы для клиента, а также базовые классы/интерфейсы для реализации сервера.
- Бизнес – логика сервера, как реализация сгенерированного контракта.
- Написание клиента, которые использует сгенерированные channel и stub.

Раздел 5

Как gRPC закрывает типичные боли REST

Protocol Buffers: что это и зачем отдельный язык описания

Protobuf описывает структуру данных декларативно и компилируется в эффективное бинарное представление. Это уменьшает размер сообщений и стоимость сериализации по сравнению с JSON.

Дополнительный эффект: появляются гарантии на уровне типов и генерации кода.

.proto – это не «документация», а исполняемый контракт для генераторов.

Вы описываете сообщения и сервисы, а компилятор ``protoc`` (и плагины вроде ``grpc-java``) производят артефакты под выбранные языки. Поэтому protobuf одновременно является форматом сообщений для обмена и формой API-контракта.

Сообщения в proto3 имеют понятную модель полей: каждое поле имеет номер, тип и кардинальность (optional/repeated и т.д.).

Номера полей — ключ к обратной/прямой совместимости: клиенты могут игнорировать неизвестные поля, а сервер может читать только то, что понимает.

Вы можете добавлять новые поля с новыми номерами, и старые клиенты обычно останутся рабочими: они просто не будут читать новые поля. Удаление полей – опасная операция: образовавшийся незанятый номер часто нужно резервировать, чтобы никто его не переиспользовал и, тем самым не сломал старых клиентов.

Менять тип поля с тем же номером почти всегда несовместимо.

HTTP/2 и мультиплексирование

HTTP/2 позволяет мультиплексировать множество RPC, (создавать виртуальные соединения в рамках одного физического TCP-соединения), тем самым снижая накладные расходы на установление множества параллельных HTTP/1.1 соединений. Это дает огромное преимущество для микросервисного взаимодействия, где происходит много мелких вызовов.

Нужно оговориться: если бизнес-логика тяжёлая или база тормозит, HTTP/2 не спасёт.

Но он убирает множество транспортных ограничений, которые существуют в REST API.

Генерация кода

Сгенерированные классы сообщений дают явные getters/setters/builders, а stubs задают форму клиентского вызова.

Таким образом избавляет API от человеческого фактора: абсолютно все, что касается контракта взаимодействия будет сгенерировано в каждом микросервисе одинаково, расхождений быть не может.

А разработчику остается только описать бизнес-логику в нужном месте, не занимаясь скучным написанием одной и той же инфраструктуры.

Одиночные и потоковые запросы

REST чаще всего моделируется как «один запрос – один ответ» на уровне HTTP, а streaming поверх HTTP встречается, но как отдельная надстройка (SSE/WebSocket).

С «один запрос – один ответ» в gRPC все также, как в REST.

А вот streaming – это нативная часть gRPC: серверный, клиентский и двунаправленный потоки описываются прямо в .proto.

Streaming используется для решения следующих задач: прогресс длительной операции, передача больших объёмов чанками, событийные потоки, репликация логов, потоковая обработка.

Итого в gRPC существуют 4 модели взаимодействия.

- Одиночный: 1 запрос - 1 ответ.
- Поток сервера: 1 запрос - много ответов.
- Поток клиента: много запросов - 1 ответ.
- Двунаправленный поток: много запросов – много ответов.

Цепочка вызовов для одиночного запроса

1. Клиент вызывает метод stub (например, blocking unary). Stub формирует сообщение запроса в формате protobuf.
2. Канал (channel) передаёт RPC поверх HTTP/2.
3. На сервере срабатывает сгенерированная «скелетная» точка входа: ваша реализация бизнес-логики формирует и отдаёт ответ через StreamObserver.
4. Ответ сериализуется обратно в protobuf и доходит до клиента, (клиентский API обычно выглядит как обычный вызов метода, скрывая сеть).

Раздел 6

Gradle: плагины, зависимости, генерация

Gradle-конфигурация

Необходимые плагины и зависимости:

```
import com.google.protobuf.gradle.id

plugins {
    ...
    id("com.google.protobuf") version "0.9.6"
}

dependencies {
    ...
    implementation(platform("io.grpc:grpc-bom:1.68.2"))
    implementation("org.springframework.grpc:spring-grpc-server-spring-boot-starter:1.0.2")
    runtimeOnly("io.grpc:grpc-services")
    implementation("com.google.protobuf:protobuf-java:4.34.1")
}
```

Важно:

Тут представлена конфигурация gradle для сервера.

Для клиента нужно использовать Spring-starter: `implementation("org.springframework.grpc:spring-grpc-client-spring-boot-starter:1.0.2")`

Файл: build.gradle.kts (сервер) (продолжение 5/8)

Настройка protobuf и sourceSets:

```
protobuf {  
    protoc {  
        artifact = "com.google.protobuf:protoc:4.34.1"  
    }  
    plugins {  
        id("grpc") {  
            artifact = "io.grpc:protoc-gen-grpc-java:1.68.2"  
        }  
    }  
    generateProtoTasks {  
        ofSourceSet("main").forEach {  
            it.plugins {  
                id("grpc")  
            }  
        }  
    }  
}  
sourceSets {  
    main {  
        java {  
            srcDirs(  
                "build/generated/sources/proto/main/grpc",  
                "build/generated/sources/proto/main/java"  
            )  
        }  
    }  
}
```

Раздел 7

Контракт: calculator.proto

Файл: src/main/proto/calculator.proto

Сервис **Calculator** объявляет RPC **Multiply** и типы сообщений запроса/ответа. Опции **java_package** и **java_multiple_files** влияют на то, как Java-код будет организован после генерации: пакеты и разбиение по файлам.

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "com.example.grpc.proto";
option java_outer_classname = "CalculatorProto";

package calculator;

service Calculator {
  rpc Multiply(MultiplyRequest) returns (MultiplyResponse);
}

message MultiplyRequest {
  int32 first_number = 1;
  int32 second_number = 2;
}

message MultiplyResponse {
  int32 result = 1;
}
```

Раздел 8

Сгенерированный код по контракту

Почему MultiplyRequest.java огромный, хотя в .proto всего пара полей

Генератор создаёт полноценную реализацию сообщения protobuf: builders, внутренние поля, сериализацию, merge, hashCode/equals (в зависимости от версии), дескрипторы и т.д. Поэтому «учебный» proto маленький, а Java-файл — большой.

На слайде с кодом показаны только информативные части: объявление типа, геттеры полей, фабрики newBuilder, сборка сообщения и сеттеры в Builder.

Сгенерированный MultiplyRequest.java

```
package com.example.grpc.proto;

/**
 * @com.google.protobuf.Generated
 * public final class MultiplyRequest extends
 *     com.google.protobuf.GeneratedMessage implements
 *     // @@protoc_insertion_point(message_implements:calculator.MultiplyRequest)
 *     MultiplyRequestOrBuilder {
 * private static final long serialVersionUID = 0L;

    public static final int FIRST_NUMBER_FIELD_NUMBER = 1;
    private int firstNumber_ = 0;

    @java.lang.Override
    public int getFirstNumber() {
        return firstNumber_;
    }

    public static final int SECOND_NUMBER_FIELD_NUMBER = 2;
    private int secondNumber_ = 0;

    @java.lang.Override
    public int getSecondNumber() {
        return secondNumber_;
    }

    public static Builder newBuilder() {
        return DEFAULT_INSTANCE.toBuilder();
    }
}
```

```
    public static Builder newBuilder(com.example.grpc.proto.MultiplyRequest
prototype) {
    return DEFAULT_INSTANCE.toBuilder().mergeFrom(prototype);
}

    @java.lang.Override
    public Builder toBuilder() {
    return this == DEFAULT_INSTANCE
        ? com.example.grpc.proto.MultiplyRequest build() {
            com.example.grpc.proto.MultiplyRequest result = buildPartial();
            if (!result.isInitialized()) {
                throw new UninitializedMessageException(result);
            }
            return result;
        }

        : new Builder setFirstNumber(int value) {
            firstNumber_ = value;
            bitField0_ |= 0x00000001;
            onChanged();
            return this;
        }

        : new Builder setSecondNumber(int value) {
            secondNumber_ = value;
            bitField0_ |= 0x00000002;
            onChanged();
            return this;
        }
    }
```

Что такое `CalculatorGrpc.java` и зачем он клиенту и серверу

В этом файле обычно находятся дескрипторы методов, имена сервисов, фабрики stub-ов (`newStub`, `newBlockingStub` и др.).

Серверная сторона наследует `CalculatorImplBase`, клиентская — использует stub для вызова.

Фрагмент ниже показывает начало файла: `SERVICE_NAME`, дескриптор unary-метода и фабрики stub-ов.

Сгенерированный CalculatorGrpc.java

Здесь представлено начало файла и метод для сервера:

getMultiplyMethod()

```
package com.example.grpc.proto;
import static io.grpc.MethodDescriptor.generateFullMethodName;

@javax.annotation.Generated(
    value = "by gRPC proto compiler (version 1.68.2)",
    comments = "Source: calculator.proto")
@io.grpc.stub.annotations.GrpcGenerated
public final class CalculatorGrpc {
    private CalculatorGrpc() {}

    public static final java.lang.String SERVICE_NAME = "calculator.Calculator";

    private static volatile
io.grpc.MethodDescriptor<com.example.grpc.proto.MultiplyRequest,
    com.example.grpc.proto.MultiplyResponse> getMultiplyMethod;

    @io.grpc.stub.annotations.RpcMethod(
        fullMethodName = SERVICE_NAME + '/' + "Multiply",
        requestType = com.example.grpc.proto.MultiplyRequest.class,
        responseType = com.example.grpc.proto.MultiplyResponse.class,
        methodType = io.grpc.MethodDescriptor.MethodType.UNARY)
    public static io.grpc.MethodDescriptor<com.example.grpc.proto.MultiplyRequest,
        com.example.grpc.proto.MultiplyResponse> getMultiplyMethod() {
        io.grpc.MethodDescriptor<com.example.grpc.proto.MultiplyRequest,
com.example.grpc.proto.MultiplyResponse> getMultiplyMethod;
        if ((getMultiplyMethod = CalculatorGrpc.getMultiplyMethod) == null) {
```

```
synchronized (CalculatorGrpc.class) {
            if ((getMultiplyMethod = CalculatorGrpc.getMultiplyMethod) == null) {
                CalculatorGrpc.getMultiplyMethod = getMultiplyMethod =
                    io.grpc.MethodDescriptor.<com.example.grpc.proto.MultiplyRequest,
com.example.grpc.proto.MultiplyResponse>newBuilder()
                        .setType(io.grpc.MethodDescriptor.MethodType.UNARY)
                        .setFullMethodName(generateFullMethodName(SERVICE_NAME, "Multiply"))
                        .setSampledToLocalTracing(true)
                        .setRequestMarshaller(io.grpc.protobuf.ProtoUtils.marshaller(
com.example.grpc.proto.MultiplyRequest.getDefaultInstance()))
                        .setResponseMarshaller(io.grpc.protobuf.ProtoUtils.marshaller(
com.example.grpc.proto.MultiplyResponse.getDefaultInstance()))
                        .setSchemaDescriptor(new
CalculatorMethodDescriptorSupplier("Multiply"))
                        .build();
            }
        }
    }
    return getMultiplyMethod;
}
```

Сгенерированный CalculatorGrpc.java

Здесь представлены stubs для клиента: `getMultiplyMethod()`

```
/**
 * Creates a new async stub that supports all call types for the service
 */
public static CalculatorStub newStub(io.grpc.Channel channel) {
    io.grpc.stub.AbstractStub.StubFactory<CalculatorStub> factory =
        new io.grpc.stub.AbstractStub.StubFactory<CalculatorStub>() {
            @java.lang.Override
            public CalculatorStub newStub(io.grpc.Channel channel, io.grpc.CallOptions callOptions) {
                return new CalculatorStub(channel, callOptions);
            }
        };
    return CalculatorStub.newStub(factory, channel);
}

/**
 * Creates a new blocking-style stub that supports unary and streaming output calls on the service
 */
public static CalculatorBlockingStub newBlockingStub(io.grpc.Channel channel) {
    io.grpc.stub.AbstractStub.StubFactory<CalculatorBlockingStub> factory = new io.grpc.stub.AbstractStub.StubFactory<CalculatorBlockingStub>() {
        @java.lang.Override
        public CalculatorBlockingStub newStub(io.grpc.Channel channel, io.grpc.CallOptions callOptions) {
            return new CalculatorBlockingStub(channel, callOptions);
        }
    };
    return CalculatorBlockingStub.newStub(factory, channel);
}
```

Сгенерированный CalculatorGrpc.java

```
/**
 * Creates a new ListenableFuture-style stub that supports unary calls on the service
 */
public static CalculatorFutureStub newFutureStub(io.grpc.Channel channel) {
    io.grpc.stub.AbstractStub.StubFactory<CalculatorFutureStub> factory =
        new io.grpc.stub.AbstractStub.StubFactory<CalculatorFutureStub>() {
            @java.lang.Override
            public CalculatorFutureStub newStub(io.grpc.Channel channel, io.grpc.CallOptions callOptions) {
                return new CalculatorFutureStub(channel, callOptions);
            }
        };
    return CalculatorFutureStub.newStub(factory, channel);
}
```

Раздел 9

Сервер: реализация RPC

CalculatorService

CalculatorService – это уже написанный разработчиком класс в проекте, который использует сгенерированный код, дописывает бизнес-логику и таким образом непосредственно реализует gRPC – сервер.

Метод multiply получает уже распаршенный MultiplyRequest и StreamObserver для ответа. В одиночном ответе обычно вызывают onNext один раз и затем onCompleted.

Аннотация Spring @GrpcService подключает bean к серверу gRPC в Spring-контексте, говоря: «этот класс — gRPC endpoint».

```
package org.example.service.grpc;

import com.example.grpc.proto.CalculatorGrpc;
import com.example.grpc.proto.MultiplyRequest;
import com.example.grpc.proto.MultiplyResponse;
import io.grpc.stub.StreamObserver;
import org.springframework.grpc.server.service.GrpcService;

@GrpcService
public class CalculatorService extends CalculatorGrpc.CalculatorImplBase {

    @Override
    public void multiply(MultiplyRequest request, StreamObserver<MultiplyResponse> responseObserver) {
        int product = request.getFirstNumber() * request.getSecondNumber();

        MultiplyResponse response = MultiplyResponse.newBuilder()
            .setResult(product)
            .build();

        responseObserver.onNext(response);
        responseObserver.onCompleted();
    }
}
```

Раздел 10

Клиент: channel + stub

Что делает клиент в этом проекте

GrpcChannelFactory создаёт канал к calculator-server. Далее создаётся CalculatorBlockingStub, который удобен для простых одиночных вызовов в синхронном стиле. Метод multiply(int a, int b) собирает protobuf-сообщение запроса и вызывает RPC, отправляя запрос на сервер.

application.yml:

```
grpc:
  client:
    channels:
      calculator-server:
        address: localhost:9090      # адрес вашего gRPC-сервера
```

CalculatorClientService

```
package org.example.server;

import com.example.grpc.proto.CalculatorGrpc;
import com.example.grpc.proto.MultiplyRequest;
import com.example.grpc.proto.MultiplyResponse;
import org.springframework.grpc.client.GrpcChannelFactory;
import org.springframework.stereotype.Service;

@Service
public class CalculatorClientService {

    private final CalculatorGrpc.CalculatorBlockingStub calculatorStub;

    public CalculatorClientService(GrpcChannelFactory channelFactory) {
        var channel = channelFactory.createChannel("calculator-server");
        this.calculatorStub = CalculatorGrpc.newBlockingStub(channel);
    }

    public int multiply(int a, int b) {
        MultiplyRequest request = MultiplyRequest.newBuilder()
            .setFirstNumber(a)
            .setSecondNumber(b)
            .build();
        MultiplyResponse response = calculatorStub.multiply(request);
        return response.getResult();
    }
}
```